EVIDENCIA DE DESPLIEGUE

Proyecto Entre Platos y Copas

APT122 · Capstone

Fase	Fase 2
Integrantes	Helen Cortez y Benjamín Romero
Sección	002D
Aplicación publicada	https://epyc-in7j.onrender.com
Versión desplegada	Commit 457425e · rama main

1. Objetivo

El objetivo de esta evidencia es demostrar la publicación del sistema Entre Platos y Copas en un entorno accesible mediante internet, incluyendo la configuración del servicio web, la conexión segura con la base de datos, el proceso de construcción y la verificación de su funcionamiento en producción.

Resumen

Servicio	Estado	Versión	Servidor	Base de datos
Render · Web Service	**Live**	457425e	Gunicorn 26.2.0	PostgreSQL 17 · Supabase

2. Arquitectura del despliegue

```
Usuario (navegador)
  ↓ HTTPS
Render · Web Service «Epyc»
  ↓
Gunicorn → Aplicación Django (+ WhiteNoise para los estáticos)
  ↓ SQL con SSL
PostgreSQL 17 en Supabase
```

- **Código:** almacenado en GitHub (https://github.com/Benjamin-Romero-Cortes/Epyc), rama main.
- **Construcción:** Render obtiene el código del repositorio, instala las dependencias y reúne los archivos estáticos.
- **Ejecución:** Django se ejecuta mediante Gunicorn; WhiteNoise entrega los archivos estáticos con versión.
- **Datos:** la información se almacena en PostgreSQL (Supabase), con conexión cifrada.
- **Secretos:** se configuran mediante variables de entorno de Render; no están en el código.



Figura 1. Sistema publicado: respuesta HTTPS, redirección y certificado (01_URL_Sistema_Desplegado.png)

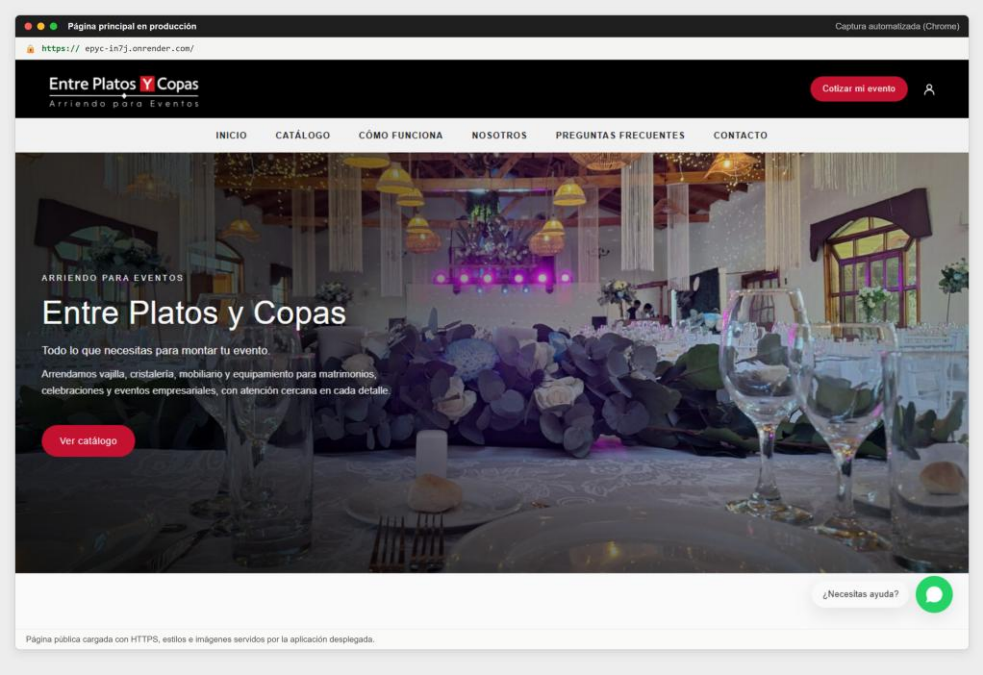


Figura 2. Página principal del sistema en producción (02_Pagina_Principal_Produccion.png)

3. Repositorio, rama y servicio

Tipo de servicio	Web Service (plan gratuito)
Entorno de ejecución	Python 3 (versión administrada por Render)
Región	Oregón (US West)
Repositorio conectado	Benjamin-Romero-Cortes/Epyc
Rama desplegada	main
Último commit desplegado	457425e · «Imagen predeterminada, ficha pública y teléfonos ordenados»
Despliegue	5 de noviembre de 2026 · iniciado manualmente desde el panel
Enlace al commit	https://github.com/Benjamin-Romero-Cortes/Epyc/commit/457425e

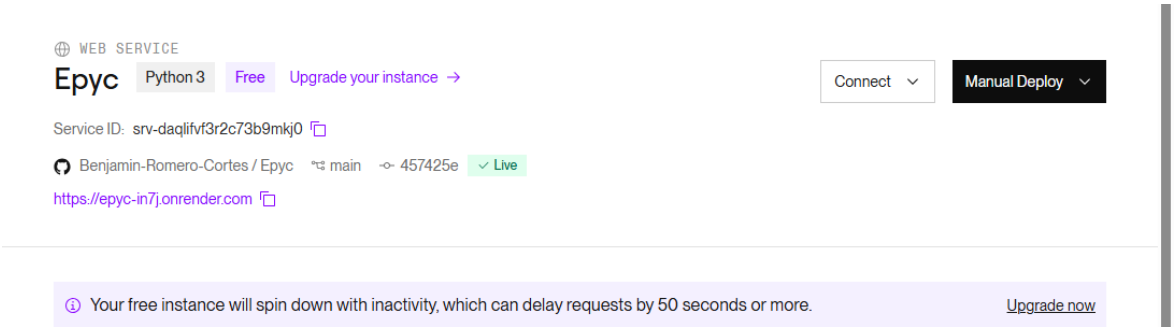


Figura 3. Servicio «Epyc» en Render: repositorio, rama, commit y estado Live (03_Servicio_Render.png)

4. Configuración de construcción e inicio

Comandos configurados en Render (Settings → Build y Deploy):

```
Build Command:   pip install -r requirements.txt && python manage.py collectstatic --noinput
Start Command:   gunicorn Epyc.wsgi:application
Pre-Deploy:      (vacío)
Root Directory:  (vacío; se usa la raíz del repositorio)
```

Comando	Para qué sirve
pip install -r requirements.txt	Instala las dependencias con las versiones fijadas del proyecto.
python manage.py collectstatic --noinput	Reúne los archivos estáticos (CSS, JavaScript e imágenes) para que WhiteNoise los entregue con versión.
gunicorn Epyc.wsgi:application	Inicia el servidor de producción que ejecuta la aplicación Django.

Las migraciones no forman parte de los comandos de Render: se aplican manualmente con python manage.py migrate sobre la base de Supabase. Render tiene el despliegue automático configurado «On Commit»; en la práctica, los 29 despliegues del historial se ejecutaron manualmente desde el panel.

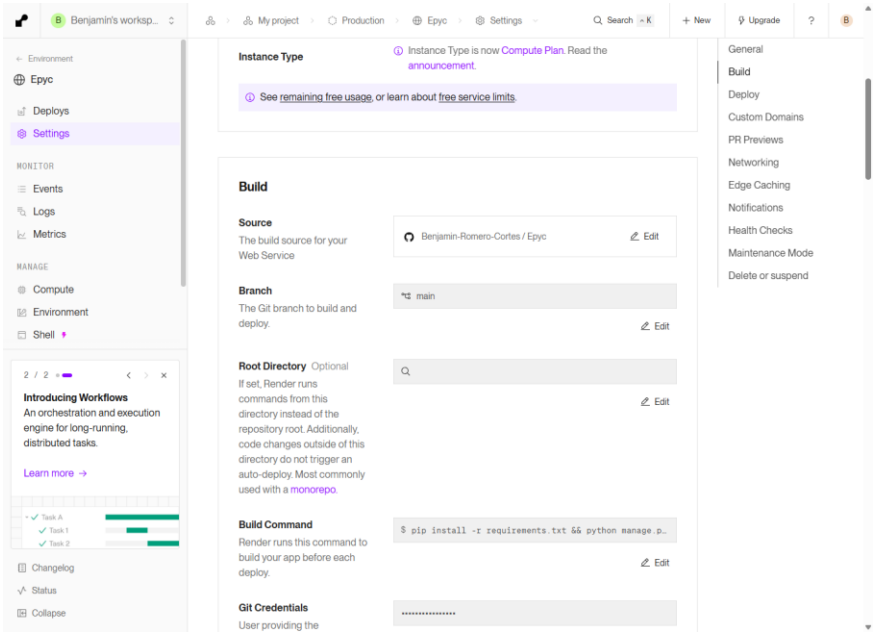


Figura 4. Configuración de construcción: origen, rama y comando de construcción (04_Configuracion_Construccion.png)

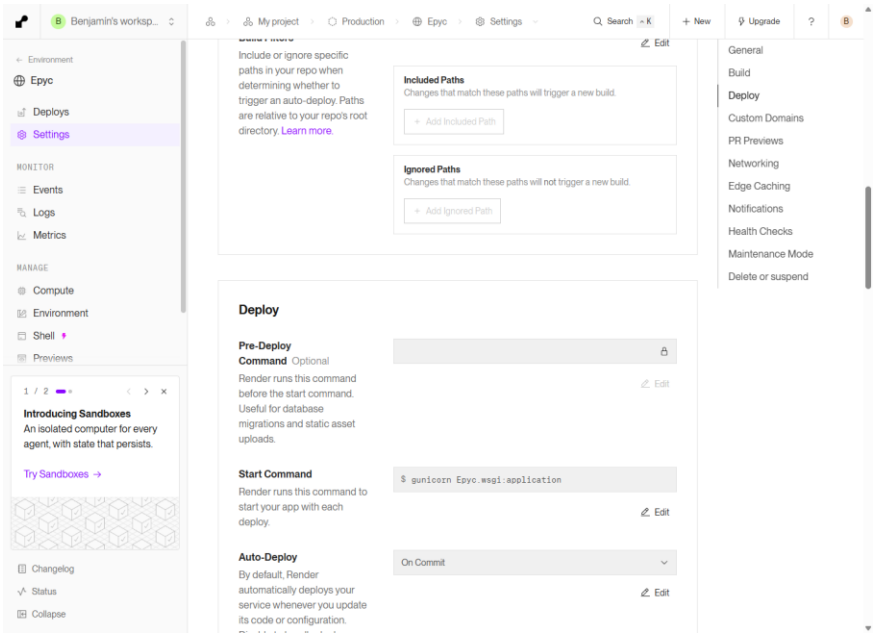


Figura 5. Comando de inicio y despliegue automático (05_Comando_Inicio_Auto_Deploy.png)

El campo del comando de construcción se recorta en pantalla por su largo; el texto completo está en el recuadro de arriba.

5. Dependencias de producción

Del archivo requirements.txt del repositorio, las dependencias relevantes para el despliegue son:

Dependencia	Uso en producción
Django 6.1.1	Framework web: rutas, vistas, plantillas, ORM y migraciones.
gunicorn	Servidor WSGI de producción (versión instalada en Render: 26.2.0, según el registro de arranque).
whitenoise 6.12.0	Entrega de archivos estáticos comprimidos y con versión, sin servidor adicional.
psycopg 3.3.5 y psycopg-binary	Conector entre Django y PostgreSQL.
python-dotenv 1.2.3	Carga de variables de entorno desde .env en desarrollo.
asgiref, sqlparse, tzdata, packaging, typing_extensions	Dependencias de soporte de Django y del entorno.

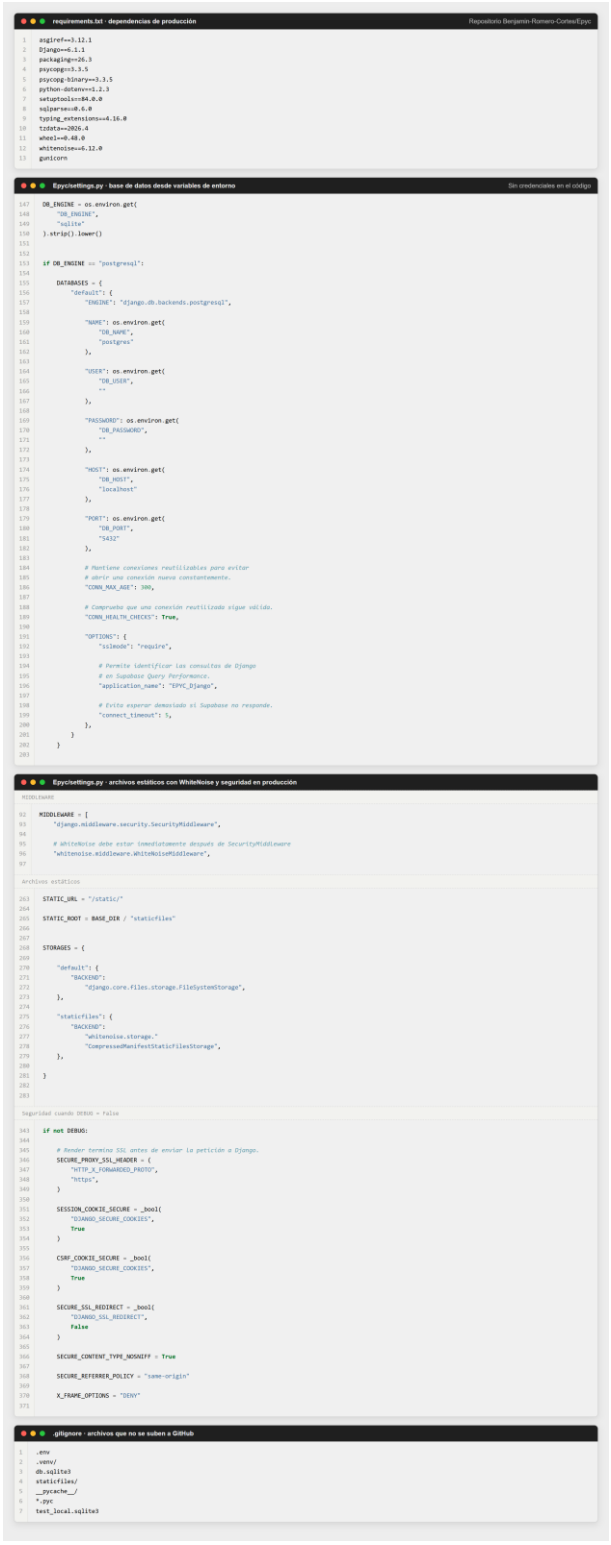


Figura 6. Archivos de configuración: dependencias, base de datos desde variables, estáticos y .gitignore
(10_Archivos_Configuracion.png)

6. Variables de entorno

El servicio tiene 12 variables de entorno. Las capturas y este documento muestran solo sus nombres; los valores están ocultos y no se incluyen credenciales ni cadenas de conexión.

Variable	Propósito
DJANGO_SECRET_KEY	Clave secreta de Django, almacenada fuera del código.
DJANGO_DEBUG	Modo de depuración. En producción vale False.
DJANGO_ALLOWED_HOSTS	Dominios que la aplicación acepta.
DJANGO_CSRF_TRUSTED_ORIGINS	Orígenes HTTPS autorizados para enviar formularios.
DB_ENGINE	Selecciona el motor de base de datos (postgresql en producción).
DB_HOST, DB_PORT, DB_NAME, DB_USER	Datos de conexión a PostgreSQL en Supabase.
DB_PASSWORD	Contraseña de la base de datos.
EMAIL_HOST_USER, EMAIL_HOST_PASSWORD	Cuenta y contraseña de aplicación para enviar correos.

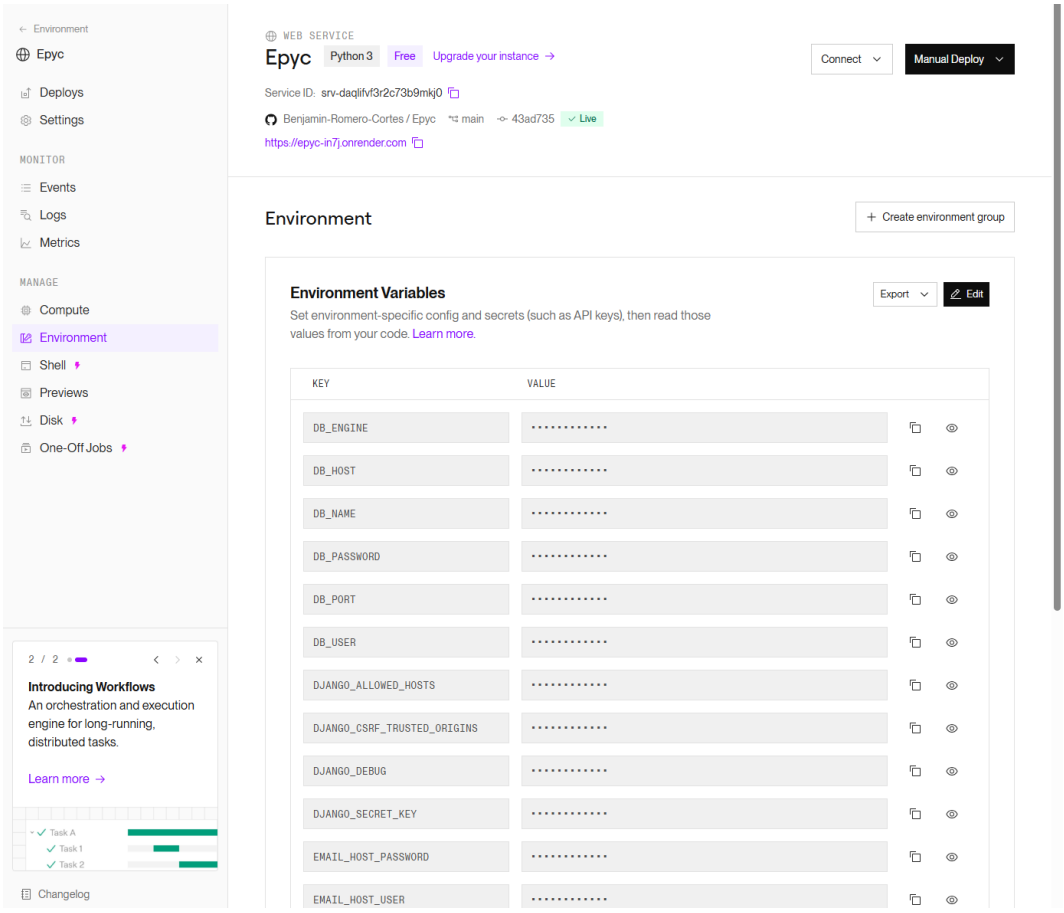


Figura 7. Variables de entorno de Render con los valores ocultos (08_Variables_Entorno.png)

7. Base de datos de producción

La aplicación publicada se conecta a PostgreSQL 17 en Supabase mediante las variables de entorno y con conexión cifrada (SSL). El detalle del modelo y de las tablas se presenta en la evidencia de base de datos; aquí solo se demuestra la conexión.

- El panel de inventario lee las existencias desde PostgreSQL.
- Las 10 migraciones del proyecto figuran aplicadas en la base de producción.
- La cotización COT-20261105-646A4D, creada durante las pruebas, permanece disponible después de recargar la página.

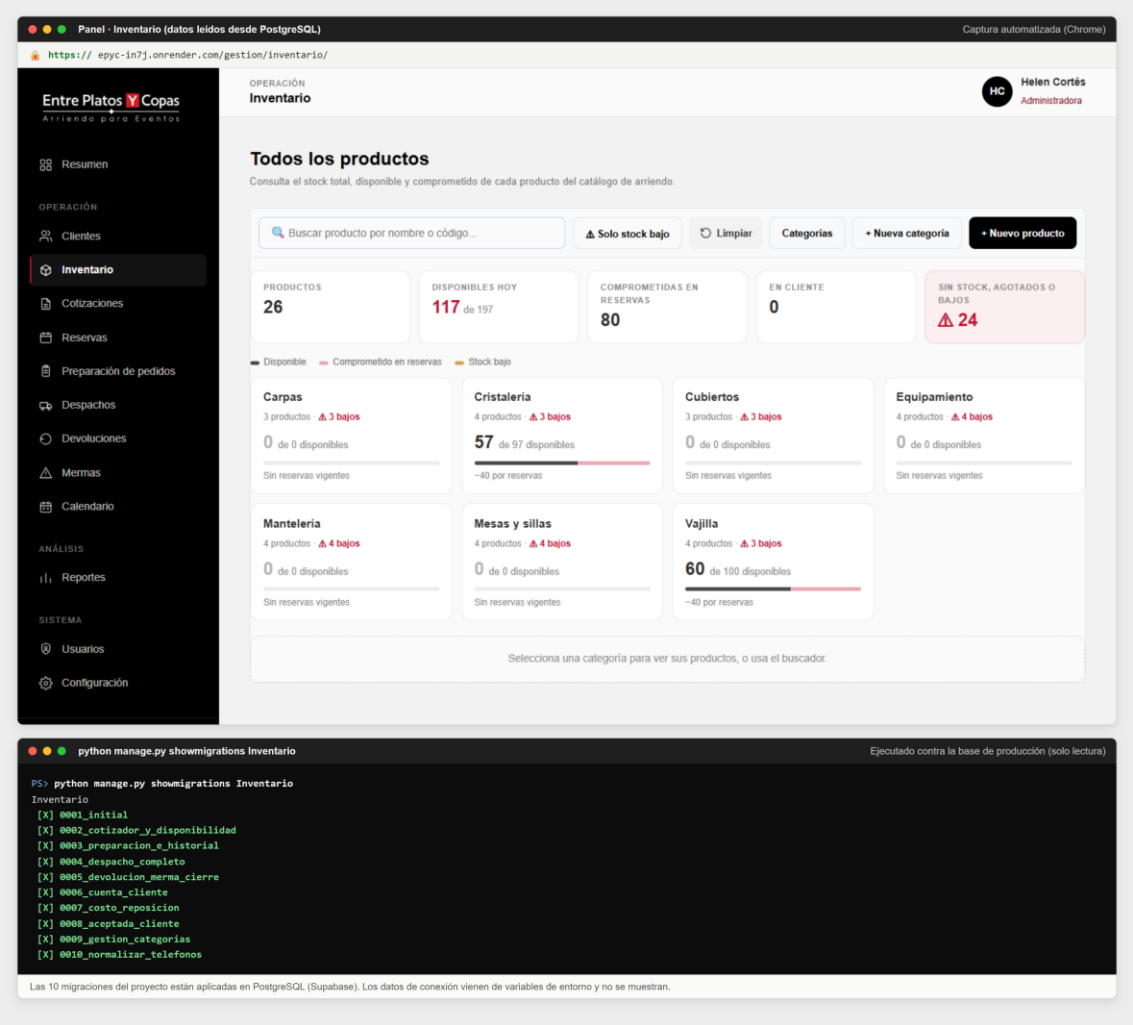


Figura 8. Inventario leído desde PostgreSQL y migraciones aplicadas (09_Conexion_Base_Datos.png)

8. Historial de despliegues

El historial de Render registra 29 despliegues. El último (commit 457425e) quedó exitoso y en estado Live. Los despliegues fallidos de los primeros días de configuración, visibles en el historial, fueron superados por despliegues posteriores exitosos.

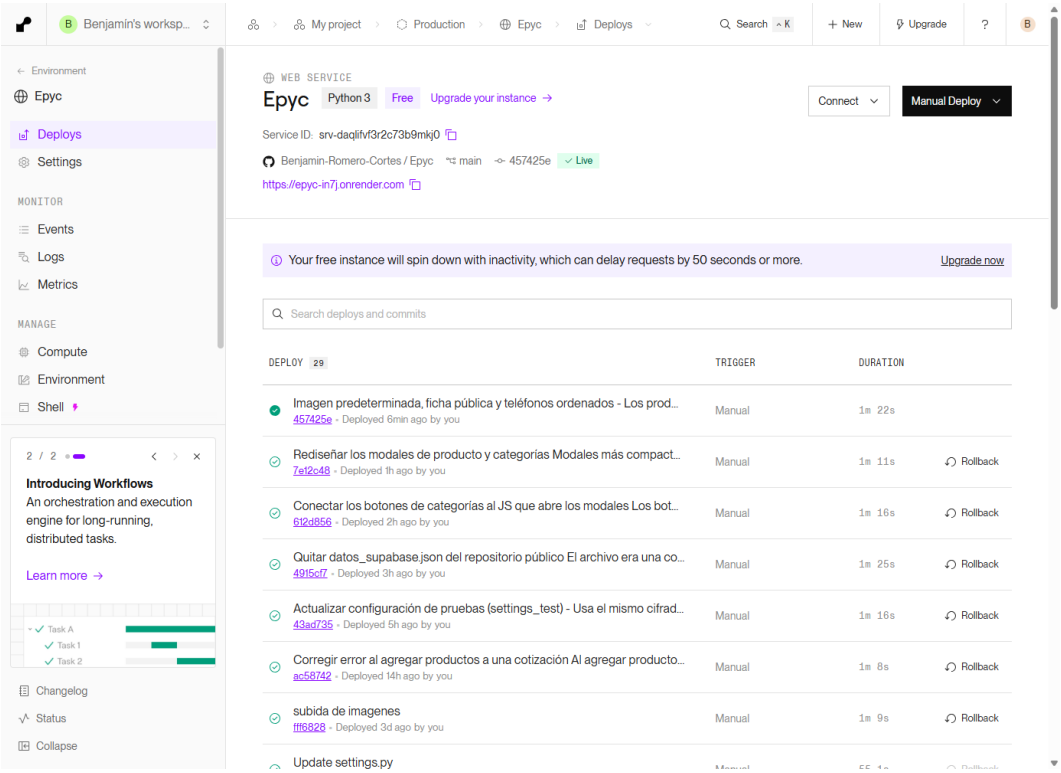


Figura 9. Historial de despliegues con el último despliegue exitoso (Live) (06_Historial_Despliegues.png)

9. Registros del servidor

Registros reales del arranque del servidor en Render (despliegue del 5 de noviembre de 2026):

```
==> Deploying...
==> Setting WEB_CONCURRENCY=1 by default, based on available CPUs in the instance
==> Running 'gunicorn Epyc.wsgi:application'
[INFO] Starting gunicorn 26.2.0
[INFO] Listening at: http://0.0.0.0:10000
[INFO] Using worker: sync
[INFO] Booting worker with pid: 59
==> Your service is live
==> Available at your primary URL https://epyc-in7j.onrender.com
```

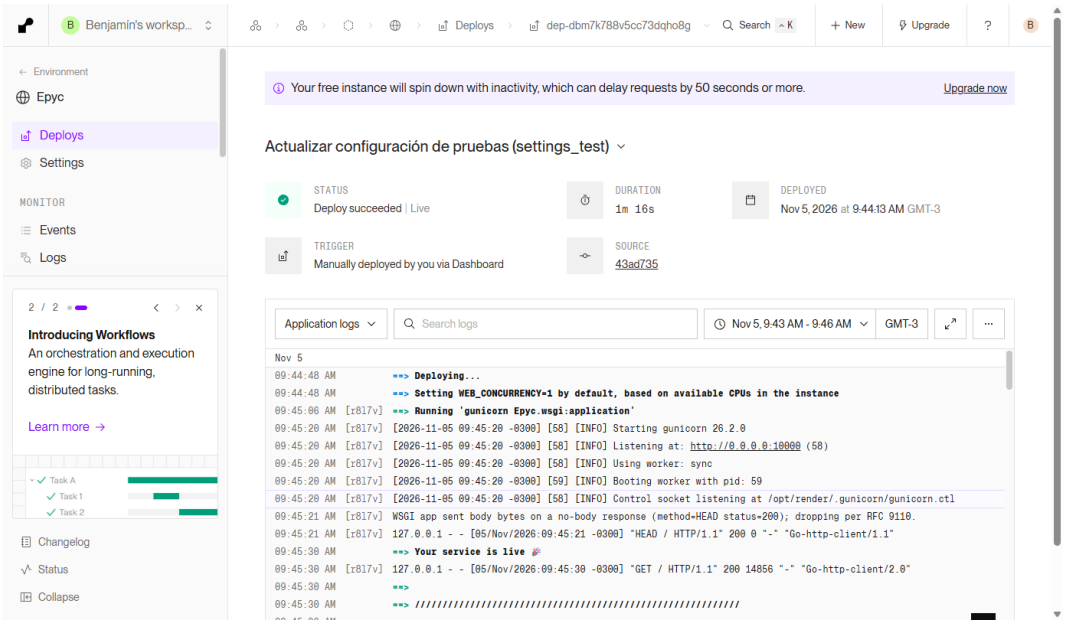


Figura 10. Registros de aplicación del arranque del servidor (07_Logs_Inicio_Servidor.png)

El panel de Render indica que los registros de construcción de ese despliegue ya no están disponibles («Build not found»), por lo que la instalación de dependencias se respalda con el comando de construcción configurado y con el resultado «Deploy succeeded». Los registros se revisaron antes de capturarlos: no contienen contraseñas, tokens ni cadenas de conexión.

10. Verificación del sistema publicado

ID	Verificación	Resultado esperado	Resultado obtenido	Estado
DEP-01	Acceso mediante URL pública	Página principal disponible	HTTP 200 en https://epyc-in7j.onrender.com	**Cumple**
DEP-02	Certificado HTTPS	Navegación segura	Certificado válido (emisor Google Trust Services, vigente hasta el 20/12/2026); HTTP redirige a HTTPS (301)	**Cumple**
DEP-03	Servicio de Render	Estado Live	Web Service «Epyc» en estado Live	**Cumple**
DEP-04	Rama y commit	Coinciden con GitHub	Rama main, commit 457425e, igual al último commit público del repositorio	**Cumple**
DEP-05	Construcción	Finaliza correctamente	Despliegue exitoso en 1 min 16 s	**Cumple**
DEP-06	Inicio con Gunicorn	Servidor operativo	Registro: «Starting gunicorn 26.2.0», «Listening at 0.0.0.0:10000» y «Your service is live»	**Cumple**
DEP-07	Archivos estáticos	CSS e imágenes cargan	Los CSS se sirven con versión en el nombre y responden 200	**Cumple**
DEP-08	PostgreSQL	La aplicación consulta y guarda datos	El panel lee de Supabase y la cotización COT-20261105-646A4D persiste tras recargar	**Cumple**
DEP-09	Migraciones	Aplicadas correctamente	Las 10 migraciones de Inventario figuran aplicadas ([X])	**Cumple**
DEP-10	Seguridad	Secretos no expuestos	DEBUG en False, variables ocultas y ninguna credencial en las capturas	**Cumple**

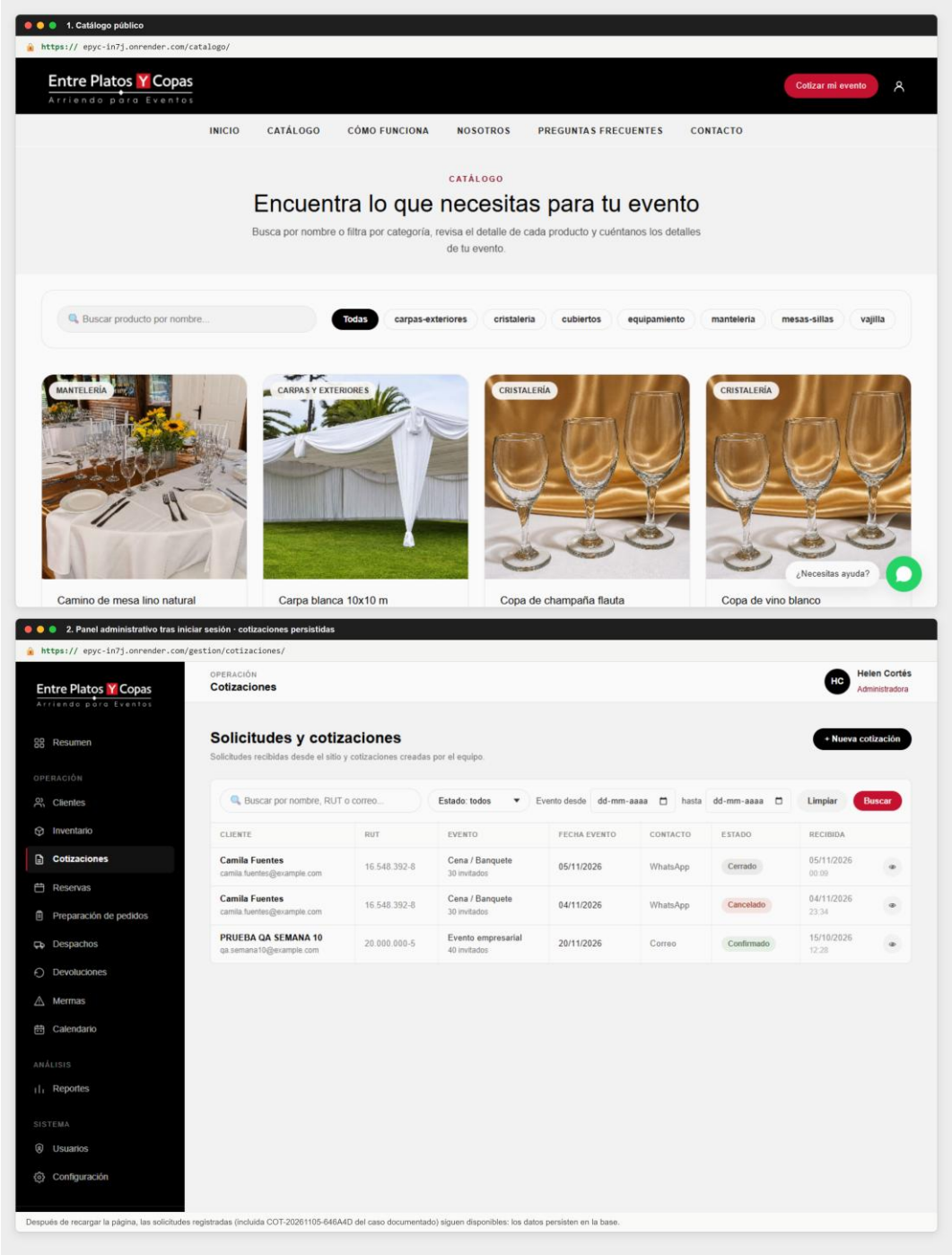


Figura 11. Catálogo público y panel administrativo con datos persistidos (12_Prueba_Final_Produccion.png)

11. Seguridad del despliegue

Medida	Estado
HTTPS	Activo; HTTP redirige a HTTPS.
DEBUG = False	Aplicado en producción. Antes de esta revisión estaba en True; se cambió la variable DJANGO_DEBUG y se desplegó nuevamente.
Claves en variables de entorno	Clave secreta, base de datos y correo se configuran en Render y no están en el código.
.env y bases locales excluidos	El archivo .gitignore excluye .env, db.sqlite3 y test_local.sqlite3.
Acceso administrativo	El panel exige inicio de sesión: sin sesión redirige al formulario de acceso.
Errores	Una ruta inexistente muestra la página de error del sitio, sin información técnica.
Evidencias	No se publican credenciales ni cadenas de conexión.



Figura 12. Comprobaciones de seguridad del sistema publicado (11_Seguridad_Produccion.png)

12. Cambio realizado durante la preparación de esta evidencia

Al revisar el servicio se detectó que DJANGO_DEBUG estaba en True: una ruta inexistente mostraba la página técnica de Django. Se cambió la variable a False en Render y se desplegó nuevamente (commit 43ad735). Antes de hacerlo se comprobó que el comando de construcción ya incluía collectstatic y que el sitio respondía con los dominios autorizados. Después se verificó que las páginas públicas, el catálogo, el cotizador, el acceso al panel y los archivos estáticos siguen funcionando. Ese despliegue también publicó los cambios pendientes de las pruebas y del formato de montos.

Después se publicó la gestión de categorías del inventario (crear, renombrar, activar, desactivar y eliminar categorías vacías) en cuatro despliegues manuales: 4915cf7 (retiro del archivo datos_supabase.json del repositorio), 612d856 (conexión de los botones con su JavaScript), 7e12c48 (rediseño de los modales) y 457425e (imagen predeterminada de productos, corrección de la ficha pública y orden de teléfonos; versión actual). La migración 0009_gestion_categorias, que agrega un índice único para el nombre de la categoría por negocio, se aplicó manualmente con python manage.py migrate sobre Supabase, antes de revisar la funcionalidad en el sitio publicado. La migración 0010_normalizar_telefonos, de datos, se aplicó del mismo modo y dejó los celulares ya guardados en el formato +56 9 1234 5678.

13. Disponibilidad y limitaciones

El servicio usa el plan gratuito de Render. Según el aviso del propio panel, la instancia se suspende tras un período de inactividad y la primera solicitud posterior puede demorar 50 segundos o más. Una vez iniciada, el sistema queda disponible mediante HTTPS. Para una evaluación conviene abrir la dirección unos minutos antes.

Conclusión

El sistema Entre Platos y Copas se encuentra desplegado y accesible mediante HTTPS. Render obtiene el código desde GitHub, instala las dependencias y ejecuta la aplicación Django mediante Gunicorn. La aplicación se conecta a PostgreSQL en Supabase mediante variables de entorno protegidas. Las verificaciones realizadas confirman que la interfaz, el acceso administrativo, la persistencia de datos y los archivos estáticos funcionan correctamente en producción.